Ψηφιακή Επεξεργασία Εικόνας - 1η εργασία

Κουδούνης Βασίλειος

ΑΕΜ:10739

Εισαγωγή

Η εργασία αυτή ασχολείται με αλγορίθμους, οι οποίοι στοχεύουν στην εξισσορρόπηση και αντιστοίχιση ιστογραμμάτων. Οι αλγόριθμοι αυτοί χωρίζονται βάση τριών διαφορετικών προσεγγίσεων.

α) greedy προσέγγιση
β) non-greedy προσέγγιση και
γ) post-disturbance προσέγγιση

Στην παρούσα αναφορά εξηγείται η λειτουργία των ζητούμενων συναρτήσεων, συμπεραλαμβανομένων και οι υλοποιήσεις των τριών προσεγγίσεων, παραθέτονται τα αποτελέσματα καθώς και ο σχολιασμός αυτών.

Συναρτήσεις - **hist-utils.py**

Calculate-Hist-Of-Img

Η συνάρτηση calculate-hist-of-img υπολογίζει το ιστόγραμμα μιας εικόνας που μπορεί να είναι σε κλίμακα [0,1] ή [0,255], ως dictionary. Στο λεξικό αυτό τα κλειδιά είναι οι τιμές pixel και οι τιμές είναι οι συχνότητες (0-255).Αν return-normalized=True, κανονικοποιεί τις συχνότητες διαιρώντας με το συνολικό αριθμό pixel, ώστε να αθροίζουν σε 1.

```python
def calculate_hist_of_img(img_array: np.ndarray, return_normalized: bool) -> Dict[int, int]:
    """Υπολογισμός ιστογράμματος για εικόνα σε [0,1] ή [0,255]."""
    #Το ιστόγραμμα υπολογίζεται για τιμές 0-255, οπότε η εικόνα πρέπει να είναι σε αυτήν την κλίμακα.
    #Αν η είσοδος είναι float ([0, 1]), θεωρείται ότι πρέπει να κλιμακωθεί.
    if np.issubdtype(img_array.dtype, np.floating):
        img_uint8 = (img_array * 255).astype(np.uint8)
    else:
        img_uint8 = img_array.astype(np.uint8)

    #Βρίσκουμε τις μοναδικές τιμές pixels και τον αριθμό εμφανίσεών τους.
    # και δημιουργούμε το dictionary
    unique, counts = np.unique(img_uint8, return_counts=True)
    hist = dict(zip(unique, counts))

    #Κανονικοποίηση
    if return_normalized:
        total = sum(counts)
        return {k: v / total for k, v in hist.items()}
    return hist
```

apply-hist-modification-transform

Η συνάρτηση apply-hist-modification-transform εφαρμόζει έναν μετασχηματισμό ιστογράμματος σε μια εικόνα, χρησιμοποιώντας ένα λεξικό που αλλάζει τιμές pixel μέσω ενός transform. Τα pixels με τιμή key αντικαθίστανται με την τιμή value, σύμφωνα με τον μετασχηματισμό που δίνεται. Τέλος, λαμβάνονται κάποια μέτρα για να διασφαλιστεί ο σωστός τύπος δεδομένων, επειδή αντιμετώπισα πολλά σφάλματα στην πορεία της εργασίας που οφείλονταν σε λάθος χειρισμό του τύπου δεδομένων.

```python
def apply_hist_modification_transform(img_array: np.ndarray, transform_dict: Dict[int, int]) -> np.ndarray:
    """Apply histogram modification transform to an image array."""
    # Convert float [0,1] -> int [0,255]
    img_levels = (img_array * 255).astype(np.uint8)

    # Initialize mapped with the same shape and data type as img_levels
    mapped = np.zeros_like(img_levels, dtype=np.uint8)

    for key, value in transform_dict.items():  #loop through dictionary
        # Ensure key and value are within the expected range
        key = int(key)  # Convert key to integer if it's not already (αποφυγή σφαλμάτων)
        value = int(value)

        # Now perform the assignment
        # Clip value to the valid range of mapped array to avoid errors
        value = np.clip(value, 0, 255)
        mapped[img_levels == key] = value

    # Return back to float [0,1]
    return mapped.astype(np.float32) / 255.0
```

Plot-image-histogram

Η συνάρτηση plot-image-histogram χρησιμοποιήθηκε για την εμφάνιση των ιστογραμμάτων των εικόνων.

```python
def plot_image_histogram(image_array: np.ndarray, title: str = "Ιστόγραμμα Εικόνας") -> None:
    """
    Συνάρτηση για την εμφάνιση του ιστογράμματος μιας εικόνας.

    Ορίσματα:
        image_array: Ο πίνακας NumPy που αναπαριστά την εικόνα.
        title: Ο τίτλος του γραφήματος.
    """
    # Επίπεδα pixel
    pixel_values = image_array.flatten()

    # Δημιουργία ιστογράμματος
    plt.hist(pixel_values, bins=256, range=(0, 255))

    # Ρυθμίσεις γραφήματος
    plt.title(title)
    plt.xlabel("Στάθμη Pixel")
    plt.ylabel("Συχνότητα")

    # Εμφάνιση γραφήματος
    plt.show()
```

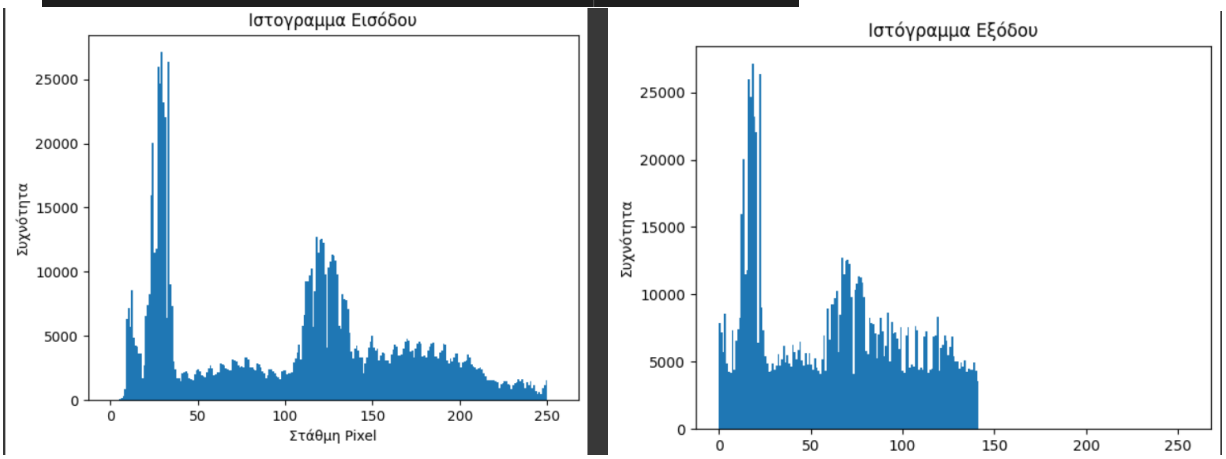
Αλγόριθμοι εξισσορρόπησης ιστογράμματος

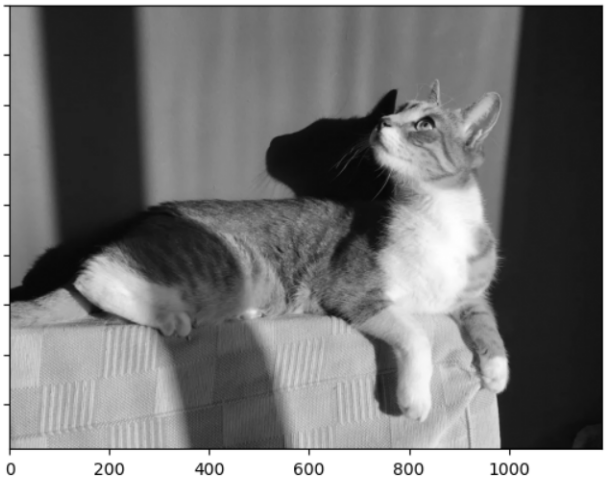
1. greedy histogram equalization

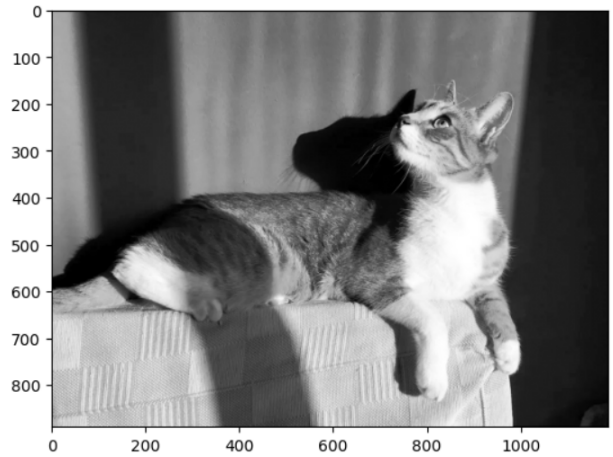
Ο στόχος είναι να αναδιανείμει τα pixel της εικόνας έτσι ώστε κάθε στάθμη εξόδου να έχει περίπου ίδιο αριθμό pixels (ισοκατανεμημένο ιστόγραμμα).

Στην περίπτωση εφαρμογής του greedy histogram equalization, παρατηρείται ότι η πλειονότητα των pixels αντιστοιχίζεται στα πρώτα bins της εξόδου. Αυτό συμβαίνει διότι ο αλγόριθμος προοδευτικά αθροίζει τις στάθμες φωτεινότητας της εισόδου μέχρι να γεμίσει το απαιτούμενο πλήθος δειγμάτων ανά bin (N/Lg). Σε εικόνες όπου το αρχικό ιστόγραμμα είναι έντονα μη ομοιόμορφο (με πολύ μεγαλύτερες συγκεντρώσεις pixels σε χαμηλές φωτεινότητες) τα output bins γεμίζουν πολύ νωρίς, κυρίως από αυτές τις χαμηλές περιοχές. Ως αποτέλεσμα, οι υψηλότερες στάθμες φωτεινότητας δεν αντιστοιχίζονται ποτέ σε ξεχωριστά bins, καθώς έχει ήδη εξαντληθεί η κατανομή των διαθέσιμων εξόδων. Ο greedy αλγόριθμος, λόγω της φύσης του να αντιστοιχίζει ολόκληρες στάθμες χωρίς δυνατότητα κατακερματισμού ή εξισορρόπησης μεταξύ πολλών bins, αδυνατεί να επιτύχει μια πιο ομοιόμορφη ισορροπία στο ιστόγραμμα εξόδου όταν το αρχικό ιστόγραμμα παρουσιάζει μεγάλες ανισότητες.

```python
def greedy_hist_equalization(hist: Dict[int, int], Lg: int) -> Dict[int, int]:
    """
    Επιστρέφει ένα λεξικό mapping των στάθμεων εισόδου σε Lg ισαπέχουσες στάθμες εξόδου,
    βασισμένο στην greedy προσέγγιση εξισορρόπησης ιστογράμματος.
    """
    N = sum(hist.values())  # συνολικά pixels
    target_per_bin = N / Lg  # στόχος ανά bin

    sorted_levels = sorted(hist.keys())
    mapping = {}

    current_bin = 0
    bin_accum = 0  # μαζεμένα pixels στο τρέχον bin

    for level in sorted_levels:
        count = hist[level]   # προσθέτουμε τα counts f0,f1,f2,...
        bin_accum += count    # για την σταθμη g0
        mapping[level] = current_bin

        if bin_accum >= target_per_bin: # Οταν επιτευχθεί ο στόχος πάμε να γεμίσουμε το επόμενο bin εξόδου
            current_bin += 1
            bin_accum = 0

            # Προστασία: να μην ξεφύγουμε πάνω από το Lg-1
            if current_bin >= Lg:
                current_bin = Lg - 1

    return mapping
```





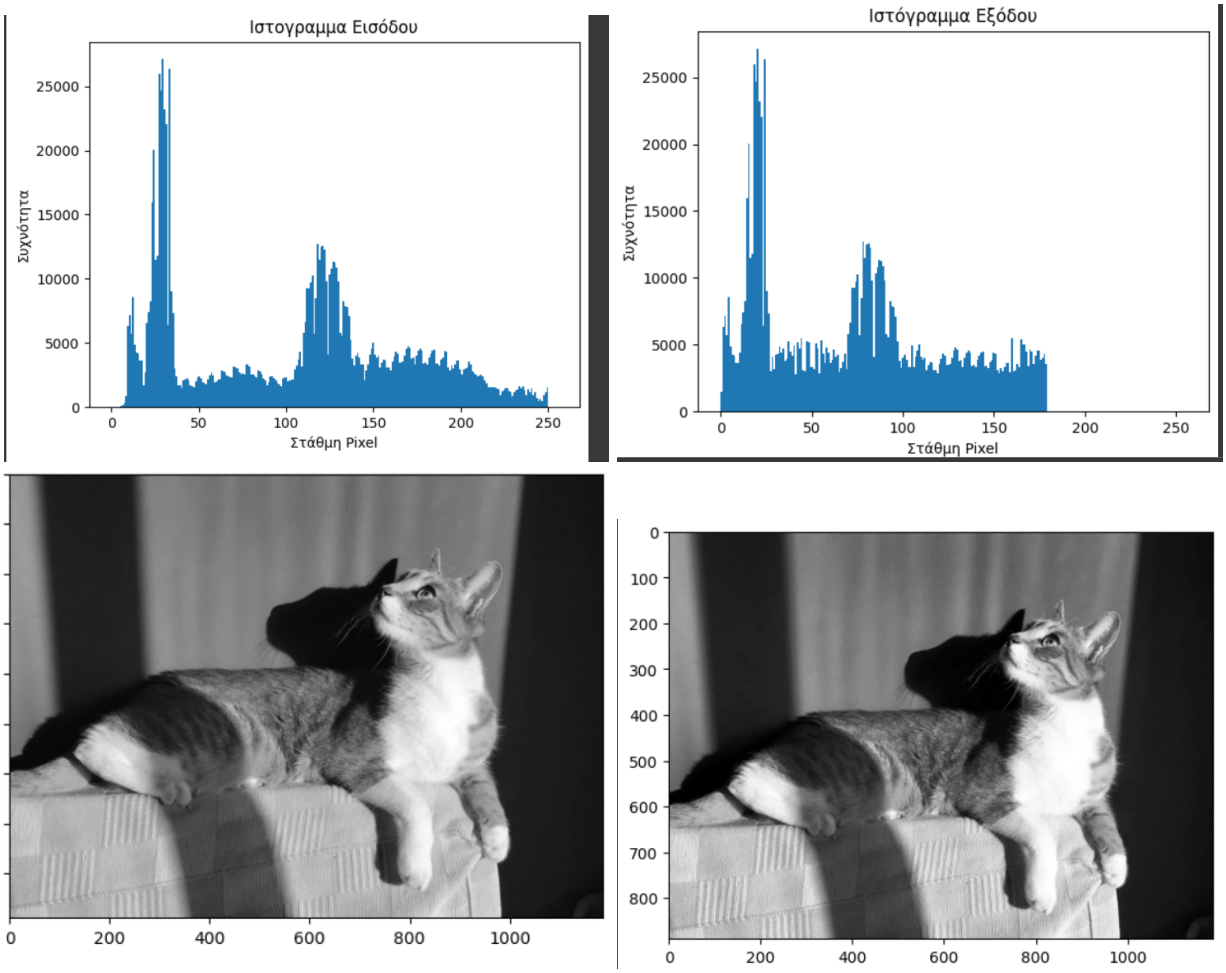


2. Non-greedy histogram equalization

Η μη-άπληστη (non-greedy) εξισορρόπηση ιστογράμματος αποτελεί μια βελτίωση σε σχέση με την απλή greedy προσέγγιση. Σε αντίθεση με τον greedy αλγόριθμο, όπου στάθμες εισόδου προστίθενται διαδοχικά μέχρι να καλυφθεί πλήρως το απαιτούμενο πλήθος δειγμάτων ανά στάθμη εξόδου, στη non-greedy μέθοδο δίνεται η δυνατότητα να σταματήσει η ανάθεση νωρίτερα, εφόσον η εναπομένουσα έλλειψη (deficiency) είναι μικρότερη ή περίπου ίση με το μισό πλήθος δειγμάτων της επόμενης υποψήφιας στάθμης εισόδου. Αυτή η λογική επιτρέπει καλύτερη διαχείριση των στάθμεων με πολύ μεγάλο πλήθος δειγμάτων, αποφεύγοντας τη συσσώρευση υπερβολικά πολλών pixels σε λίγες στάθμες εξόδου. Ως αποτέλεσμα, η μη-άπληστη εξισορρόπηση οδηγεί σε ιστόγραμμα εξόδου σημαντικά πιο ομοιόμορφο σε σχέση με το greedy, γεγονός που είναι κρίσιμο ειδικά σε εικόνες όπου το αρχικό ιστόγραμμα παρουσιάζει έντονες συγκεντρώσεις τιμών σε συγκεκριμένες περιοχές.

```python
def non_greedy_hist_equalization(hist: Dict[int, int], Lg: int) -> Dict[int, int]:
    """
    Εφαρμόζει μη-άπληστη εξισορρόπηση ιστογράμματος βάσει deficiency.
    Επιστρέφει ένα λεξικό που απεικονίζει κάθε στάθμη εισόδου σε μία στάθμη εξόδου.
    """
    N = sum(hist.values())  # Συνολικός αριθμός pixels
    target = N / Lg         # Στόχος pixels ανά στάθμη εξόδου

    sorted_levels = sorted(hist.keys())  # Τα επίπεδα φωτεινότητας ταξινομημένα
    transform = {}  # Το λεξικό που θα περιέχει το mapping εισόδου -> εξόδου

    input_idx = 0   # Δείκτης για τις στάθμες εισόδου
    output_bin = 0  # Δείκτης για τα bins εξόδου
    epsilon = 1e-6  # Μικρή ανοχή για αριθμητικά σφάλματα

    # Όσο υπάρχουν στάθμες εισόδου και δεν έχουμε γεμίσει όλα τα output bins
    while input_idx < len(sorted_levels) and output_bin < Lg:
        acc = 0         # Πόσα pixels έχουμε μαζέψει για το τρέχον bin
        assigned = False  # Σημείο ελέγχου: εκχωρήσαμε κάτι ή όχι

        # Γεμίζουμε το τρέχον bin με στάθμες εισόδου
        while input_idx < len(sorted_levels):
            level = sorted_levels[input_idx]
            count = hist[level]

            deficiency = target - acc  # Πόσα pixels μας λείπουν ακόμα
```

```python
        # Αν η έλλειψη είναι αρκετά μεγάλη σε σχέση με το επόμενο επίπεδο
        if deficiency + epsilon >= count / 2:
            transform[level] = output_bin  # Αντιστοιχίζουμε αυτή τη στάθμη
            acc += count
            input_idx += 1
            assigned = True
        else:
            break  # Σταματάμε αν δεν συμφέρει να συνεχίσουμε

    # Αν δεν εκχωρήσαμε τίποτα, αναγκαστικά παίρνουμε την επόμενη στάθμη
    if not assigned and input_idx < len(sorted_levels):
        level = sorted_levels[input_idx]
        transform[level] = output_bin
        input_idx += 1

    output_bin += 1  # Προχωράμε στο επόμενο output bin

# Ό,τι στάθμες έμειναν, τις αντιστοιχίζουμε στο τελευταίο bin
for idx in range(input_idx, len(sorted_levels)):
    transform[sorted_levels[idx]] = Lg - 1

return transform
```



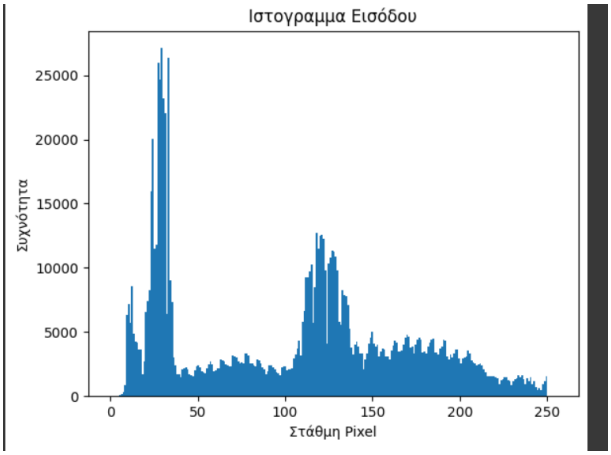
Παρατηρώντας το αρχικό ιστόγραμμα της εικόνας, διαπιστώνεται ότι υπήρχε ήδη κάλυψη σχεδόν σε όλο το εύρος φωτεινοτήτων, από 0 έως 255. Παρόλο που εμφανίζονταν έντονα peaks σε συγκεκριμένες περιοχές (γύρω στις στάθμες 30–50 και 100–140), δεν υπήρχε ουσιαστική απουσία τιμών στις υψηλότερες φωτεινότητες. Με την εφαρμογή των μεθόδων εξισορρόπησης, τόσο με την greedy όσο και με την non-greedy προσέγγιση, παρατηρείται ότι τα peaks παρέμειναν σε μεγάλο βαθμό, ενώ η συνολική κατανομή των τιμών έγινε ελαφρώς πιο ισόρροπη. Συνεπώς, μπορεί να ειπωθεί ότι το αρχικό ιστόγραμμα δεν παρουσίαζε σοβαρό πρόβλημα που να απαιτεί δραστική εξισορρόπηση, και ότι οι επεμβάσεις βελτίωσαν μεν την ισοκατανομή, αλλά χωρίς να αλλάξουν δραματικά τη συνολική μορφή του ιστογράμματος.

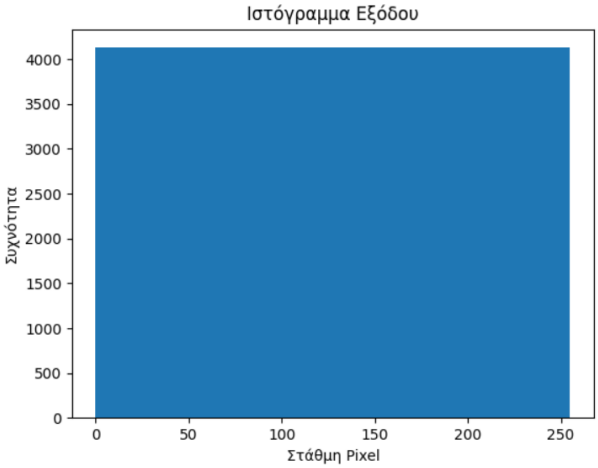
3. post-disturbance histogram equalization

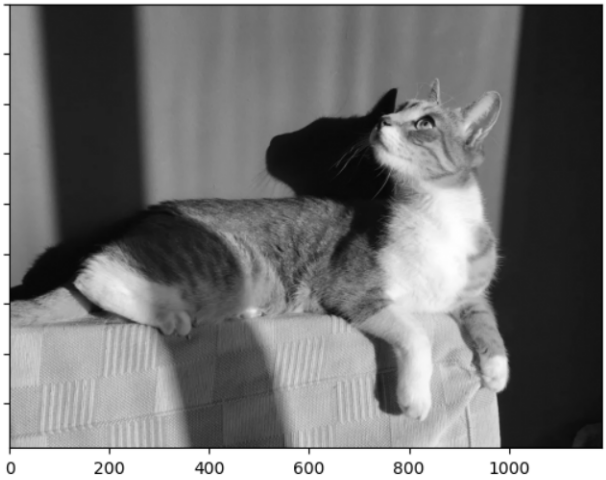
Ο στόχος είναι να κάνουμε ισοκατανομή των τιμών της εικόνας, αλλά χωρίς το πρόβλημα που προκαλείται όταν πάρα πολλά pixels έχουν ακριβώς την ίδια τιμή. Πρακτικά, ¨απλώνουμε' τα pixels σε ένα συνεχές εύρος τιμών. Ο παρακάτω αλγόριθμος υλοποιεί την εξισορρόπηση ιστογράμματος βάσει της μεθόδου post-disturbance, χρησιμοποιώντας ταξινόμηση. Αρχικά, προστίθεται μικρής έντασης τυχαίος θόρυβος (jitter) στο εύρος [0.5,+0.5] στις τιμές της εικόνας ώστε να διασπαστούν οι συγκεντρώσεις ισοδύναμων pixel τιμών. Στη συνέχεια, οι τιμές μετατρέπονται σε float στο διάστημα [0, 255] και ταξινομούνται. Η βασική ιδέα είναι να γίνει ισοκατανομή των pixel σε όλα τα δυνατά επίπεδα εξόδου (256), αποδίδοντας ίσο αριθμό pixels σε κάθε στάθμη εξόδου. Έτσι, αποφεύγεται το πρόβλημα της εκχώρησης ολόκληρων πολυπληθών στάθμεων εισόδου σε μία μόνο στάθμη εξόδου, και προκύπτει τελικά ένα σχεδόν επίπεδο και ομοιογενές ιστόγραμμα εξόδου, που αξιοποιεί πλήρως το δυναμικό εύρος της εικόνας.

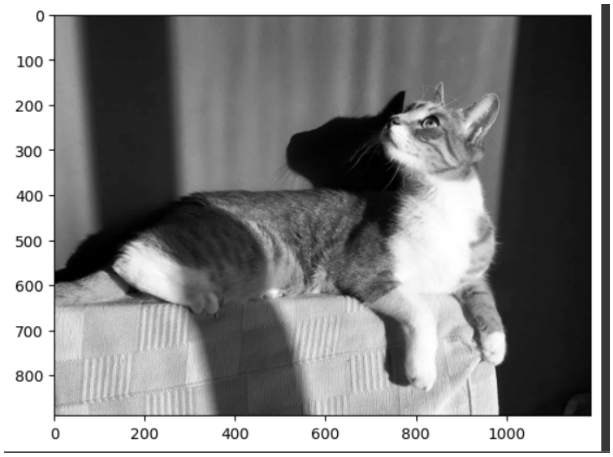
```python
def post_disturbance_equalization(img_array: np.ndarray, num_output_levels: int = 256) -> np.ndarray:
    """
    Post-disturbance εξισορρόπηση ιστογράμματος μέσω ταξινόμησης.
    - Προσθέτει jitter για διάσπαση στάθμεων.
    - Ταξινομεί όλα τα pixels.
    - Κατανέμει ισομερώς τα pixels σε όλα τα επίπεδα εξόδου.

    Args:
        img_array: εικόνα float32 σε [0, 1].
        num_output_levels: πλήθος επιπέδων εξόδου (256)

    Returns:
        output_img: εξισορροπημένη εικόνα
    """

    # 1. Προσθήκη jitter (στο εύρος [-0.5, 0.5])
    jitter = np.random.uniform(-0.5, 0.5, size=img_array.shape)
    f_hat = img_array.astype(np.float32) * 255 + jitter

    # 2. Flatten για ταξινόμηση
    flat_input = np.clip(f_hat.flatten(), 0, 255)

    # 3. Κατανομή pixels σε output levels
    total_pixels = flat_input.size
    pixels_per_level = total_pixels // num_output_levels
    remainder = total_pixels % num_output_levels
```

```python
    # Πλήθος pixels ανά επίπεδο εξόδου (π.χ. [4096,...,4096,4097,...] για τις πρώτες στάθμες)
    target_counts = np.full(num_output_levels, pixels_per_level, dtype=int)
    target_counts[:remainder] += 1

    # 4. Ταξινόμηση και κατανομή
    sorted_indices = np.argsort(flat_input)
    mapped = np.zeros_like(flat_input, dtype=np.uint8)

    idx = 0
    for level, count in enumerate(target_counts):
        mapped[sorted_indices[idx:idx+count]] = level
        idx += count

    # 5. Επαναφορά στο σχήμα της εικόνας
    return mapped.reshape(img_array.shape)
```









Ο αλγόριθμος post-disturbance για εξισορρόπηση ιστογράμματος επιτυγχάνει εντυπωσιακά επίπεδο αποτέλεσμα, με σχεδόν απόλυτα ομοιόμορφη κατανομή φωτεινοτήτων, όπως φαίνεται στο τελικό ιστόγραμμα εξόδου. Η προσθήκη jitter επιτρέπει διάσπαση των στάθμεων και η ταξινομημένη ισοκατανομή των pixels οδηγεί σε ισόρροπη κατανομή χωρίς κενά ή αιχμές. Σε σύγκριση με τις μεθόδους greedy και non-greedy, οι οποίες διατηρούν ακόμα διακυμάνσεις, το post-disturbance παρουσιάζει την πιο πλήρη και ιδανική εξισορρόπηση, προσεγγίζοντας με ακρίβεια την επιθυμητή επίπεδη κατανομή. Είναι σαφώς ανώτερη όταν ζητείται πραγματικά ισοσταθμισμένο ιστογράφημα, ειδικά σε περιπτώσεις με αρχική κατανομή που έχει έντονες συγκεντρώσεις σε περιορισμένες στάθμες.

Αλγόριθμοι - **histogram-matching**

1.**greedy histogram matching**

Η συνάρτηση greedy-hist-match υλοποιεί παραλλαγή της greedy εξισορρόπησης ιστογράμματος με στόχο την προσέγγιση μιας δοθείσας κατανομής αναφοράς. Αρχικά υπολογίζει το ιστόγραμμα της εικόνας εισόδου και το αναγκαίο πλήθος pixels για κάθε στάθμη εξόδου, σύμφωνα με την επιθυμητή κατανομή. Στη συνέχεια, εκχωρεί προοδευτικά τις στάθμες εισόδου σε στάθμες εξόδου, χρησιμοποιώντας greedy ανάθεση, ώστε να ικανοποιηθεί όσο το δυνατόν καλύτερα η αναλογία pixel-counts που ορίζει το reference histogram.

```python
def greedy_hist_match(input_array, hist_ref):
    """
    Εφαρμόζει greedy histogram matching της εικόνας input_array
    με στόχο να προσεγγίσει το δοθέν reference histogram (hist_ref).

    Args:
        input_array: εικόνα εισόδου (τιμές 0-255, τύπος uint8 ή float κλίμακας).
        hist_ref: reference histogram (dictionary {επίπεδο: πιθανότητα}).

    Returns:
        matched: εικόνα εξόδου μετά το greedy histogram matching (uint8).
    """

    # 1. Υπολογισμός ιστόγραμματος εισόδου (μη κανονικοποιημένο)
    hist_input = calculate_hist_of_img(input_array, False)

    # 2. Κανονικοποίηση του reference histogram ώστε να γίνει PDF
    pdf_ref = np.array(list(hist_ref.values())) / sum(hist_ref.values())

    # 3. Συνολικός αριθμός pixels εισόδου
    N = sum(hist_input.values())

    # 4. Υπολογισμός του στοχευόμενου αριθμού pixels για κάθε στάθμη εξόδου
    target_counts = (pdf_ref * N).astype(int)

    # 5. Δημιουργία πίνακα αντιστοίχισης input_level -> output_level
    mapping = np.zeros(256, dtype=np.uint8)
```

```python
    input_level = 0  # δείκτης στις στάθμες εισόδου

    # 6. Για κάθε στάθμη εξόδου...
    for output_level in range(len(target_counts)):
        count_needed = target_counts[output_level]  # Πόσα pixels πρέπει να αντιστοιχίσουμε σε αυτή τη στάθμη

        # 7. Όσο χρειάζονται pixels και υπάρχουν διαθέσιμα input levels...
        while count_needed > 0 and input_level < 256:
            count_available = hist_input[input_level]  # Πόσα pixels είναι διαθέσιμα στην τρέχουσα input στάθμ

            if count_available == 0:
                # Αν η τρέχουσα στάθμη εισόδου δεν έχει pixels, προχωράμε στην επόμενη
                input_level += 1
                continue

            # 8. Αναθέτουμε όσα pixels μπορούμε χωρίς να ξεπεράσουμε το ζητούμενο
            assign = min(count_needed, count_available)

            mapping[input_level] = output_level  # Αντιστοιχίζουμε το input_level στο output_level
            hist_input[input_level] -= assign    # Μειώνουμε τα διαθέσιμα pixels στο input_level
            count_needed -= assign               # Μειώνουμε τα pixels που απομένουν να καλυφθούν

            if hist_input[input_level] == 0:
                # Αν εξαντληθούν τα pixels σε αυτή τη στάθμη εισόδου, προχωράμε στην επόμενη
                input_level += 1

    # 9. Εφαρμογή του mapping: μετασχηματίζουμε την input εικόνα
    matched = mapping[input_array]

    return matched
```

✓ 2s completed at 16:05

2.**non-greedy histogram matching**

Η συνάρτηση non-greedy-hist-match υλοποιεί μη-άπληστη (non-greedy) αντιστοίχιση ιστογράμματος με στόχο την προσέγγιση μιας δοθείσας κατανομής αναφοράς. Για κάθε στάθμη εξόδου, η συνάρτηση αθροίζει διαδοχικά τις στάθμες εισόδου, εκχωρώντας όσες ικανοποιούν κριτήριο έλλειψης (deficiency), δηλαδή όταν η ποσότητα που υπολείπεται για να συμπληρωθεί ο στόχος είναι επαρκώς μεγάλη σε σχέση με το μέγεθος της επόμενης στάθμης εισόδου. Με τον τρόπο αυτό αποφεύγεται η υπερσυγκέντρωση πολλών pixels σε λίγα επίπεδα, επιτυγχάνοντας πιο ισορροπημένο histogram matching σε σύγκριση με την απλή greedy στρατηγική.

```python
def non_greedy_hist_match(input_array: np.ndarray, hist_ref: Dict[int, float]) -> np.ndarray:
    """
    Performs non-greedy histogram matching using deficiency criterion
    based on a reference histogram (hist_ref).

    Args:
        input_array: Input image array (values 0-255).
        hist_ref: Dictionary {level: pref(level)} where pref(level) is the desired probability (sums to ~1).

    Returns:
        matched_array: The output array after histogram matching.
    """
    epsilon = 1e-8

    # 1. Calculate input histogram
    hist_input = calculate_hist_of_img(input_array, return_normalized=False)

    # 2. Initialize
    N = sum(hist_input.values())  # total number of pixels
    input_levels = np.array(sorted(hist_input.keys()))
    ref_levels = np.array(sorted(hist_ref.keys()))

    # Desired counts for each output level
    desired_counts = {g: N * hist_ref[g] for g in ref_levels}

    # 3. Prepare mapping
    mapping = np.zeros(256, dtype=np.uint8)
```

```python
    # 4. Matching process
    current_input_idx = 0
    for g in ref_levels:
        accumulated_count = 0
        target_count = desired_counts[g]

        while current_input_idx < len(input_levels):
            fj = input_levels[current_input_idx]
            fj_count = hist_input[fj]

            if fj_count < epsilon:
                current_input_idx += 1
                continue

            # Deficiency calculation as per description
            deficiency = target_count - accumulated_count

            if deficiency + epsilon >= fj_count / 2 or accumulated_count == 0: #Always map first input to curren
                mapping[fj] = g
                accumulated_count += fj_count
            else:
                break #Move to the next g level

            current_input_idx += 1

    # 5. If any input levels are left unmapped, assign them to the last output level
    for idx in range(current_input_idx, len(input_levels)):
        fi = input_levels[idx]
        mapping[fi] = ref_levels[-1]

    # 6. Apply mapping
    matched_array = mapping[input_array]
    return matched_array
```

3.post-disturbance-hist-match

Στην post-disturbance μέθοδο (jittered-greedy-hist-match), η εικόνα εισόδου μετασχηματίζεται πρώτα μέσω προσθήκης μικρής κλίμακας τυχαίου θορύβου (jitter), διασπώντας τις στάθμες εισόδου σε μια πιο συνεχόμενη και πλούσια κατανομή τιμών. Έπειτα, όλα τα πιξελς ταξινομούνται και κατανέμονται greedily στα επίπεδα εξόδου, σύμφωνα με το ζητούμενο histogram αναφοράς. Με αυτόν τον τρόπο αποφεύγεται το φαινόμενο όπου πολλές ίδιες στάθμες εισόδου συγκεντρώνονται σε μια μόνο στάθμη εξόδου, οδηγώντας σε πολύ πιο ισοσταθμισμένα και ομοιόμορφα αποτελέσματα.

```python
def jittered_greedy_hist_match(input_array, hist_ref):
    # 0. Αν hist_ref είναι dict -> το μετατρέπουμε σε array
    if isinstance(hist_ref, dict):
        hist_array = np.zeros(256)
        for k, v in hist_ref.items():
            hist_array[int(k)] = v
        hist_ref = hist_array
    else:
        hist_ref = np.asarray(hist_ref)

    # 1. Προσθέτουμε jitter [-0.5, 0.5] στις τιμές της εικόνας
    jitter = np.random.uniform(-0.5, 0.5, size=input_array.shape)
    f_hat = input_array.astype(np.float32) + jitter

    # 2. Flatten για ευκολία
    flat_input = f_hat.flatten()

    # 3. Κανονικοποίηση reference ιστογράμματος
    ref_pdf = hist_ref / np.sum(hist_ref)

    # 4. Υπολογισμός στοχευόμενου πλήθους δειγμάτων για κάθε στάθμη
    total_pixels = len(flat_input)
    target_counts = np.round(ref_pdf * total_pixels).astype(int)
```

```python
    # 5. Εξασφάλιση ότι το άθροισμα είναι ακριβώς ίσο με total_pixels
    diff = total_pixels - np.sum(target_counts)
    if diff > 0:
        # Αν υπολείπονται pixels, τα προσθέτουμε στις στάθμες με τη μεγαλύτερη επιθυμητή πιθανότητα
        for i in np.argsort(-ref_pdf)[:diff]:
            target_counts[i] += 1
    elif diff < 0:
        # Αν περισσεύουν pixels, τα αφαιρούμε από τις στάθμες με τη μικρότερη πιθανότητα
        for i in np.argsort(ref_pdf)[:abs(diff)]:
            target_counts[i] -= 1

    # 6. Ταξινόμηση των εισόδων
    sorted_indices = np.argsort(flat_input)
    matched = np.zeros_like(flat_input)

    idx = 0
    for level, count in enumerate(target_counts):
        if count == 0:
            continue
        matched[sorted_indices[idx:idx+count]] = level
        idx += count

    # 7. reshape στην αρχική μορφή
    return matched.reshape(input_array.shape).astype(np.uint8)
```

hist-modif.py

perform-hist-modification

Η συνάρτηση perform-hist-modification εφαρμόζει τροποποίηση ιστογράμματος (histogram modification) σε μια εικόνα εισόδου, ώστε το τελικό της ιστόγραμμα να προσεγγίζει ένα επιθυμητό ιστογράφημα αναφοράς (hist-ref). Ο τρόπος με τον οποίο γίνεται η τροποποίηση εξαρτάται από το επιλεγμένο mode, το οποίο μπορεί να είναι "greedy", "non-greedy", "post-disturbance".

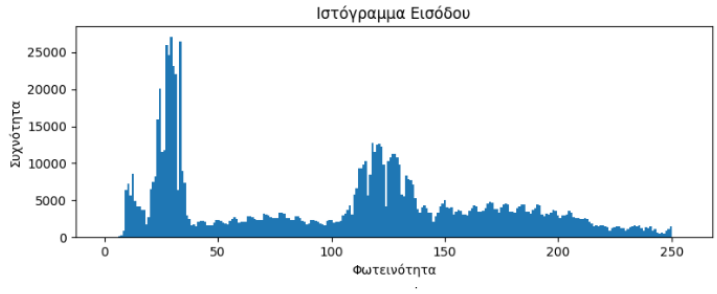
```python
def perform_hist_modification(
    img_array: np.ndarray,
    hist_ref: Dict,
    mode: str
) -> np.ndarray:
    """
    Εφαρμόζει histogram modification με βάση επιθυμητή κατανομή hist_ref και mode.
    """
    # 1. Υπολογισμός ιστογράμματος εισόδου
    input_hist = calculate_hist_of_img(img_array, return_normalized=False)

    # 2. Εξαγωγή στάθμεων και συχνοτήτων
    input_levels = np.array(list(input_hist.keys()))
    input_freq = np.array(list(input_hist.values()))
    ref_levels = np.array(list(hist_ref.keys()))
    ref_freq = np.array(list(hist_ref.values()))

    # 5. Επιλογή μεθόδου
    if mode == "greedy":

        input_array1 = (img_array * 255).clip(0, 255).astype(np.uint8)

        matched_array = greedy_hist_match(input_array1, hist_ref)

        modified_img = matched_array / 255.0
```

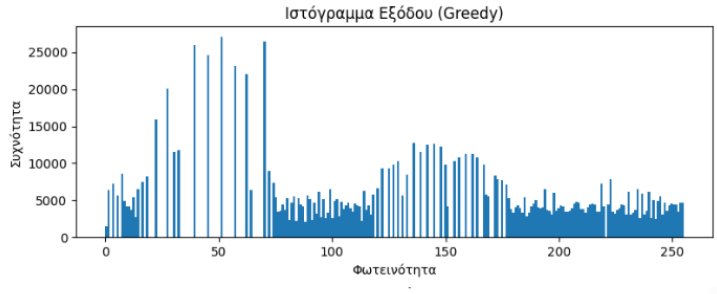
```python
    elif mode == "non-greedy":

        input_array1 = (img_array * 255).clip(0, 255).astype(np.uint8)

        print(f"hist_ref = {hist_ref}")
        matched_array = improved_hist_match_from_hist2(input_array1, hist_ref)

        modified_img = matched_array / 255.0  # <-- γύρνα στο [0,1]



    elif mode == "post-disturbance":
        input_array1 = (img_array * 255).clip(0, 255).astype(np.uint8)

        modified_img = jittered_greedy_hist_match(input_array1, hist_ref).astype(np.uint8)

        modified_img = modified_img / 255.0



    else:
        raise ValueError("Άγνωστη μέθοδος. Επιλέξτε από: 'greedy', 'non-greedy', 'post-disturbance'")

    return modified_img
```

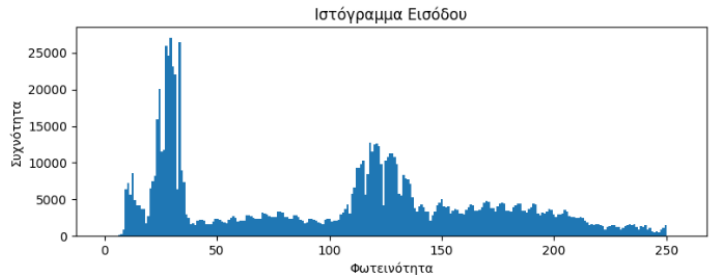
perform-hist-eq

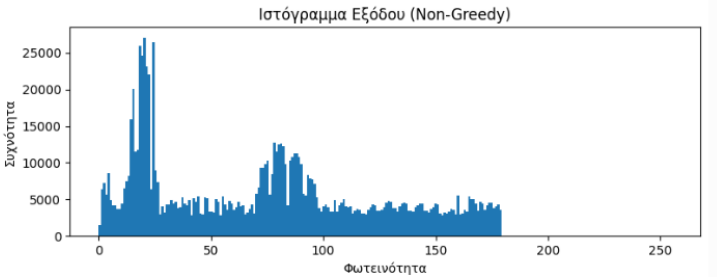
Η συνάρτηση perform-hist-eq εφαρμόζει εξισορρόπηση ιστογράμματος (histogram equalization) σε μια είσοδο εικόνα img-array, χρησιμοποιώντας ως επιθυμητό histogram μια ισοκατανεμημένη κατανομή (δηλαδή ένα ομοιόμορφο ιστόγραμμα). Ο τρόπος με τον οποίο θα γίνει η ισοστάθμιση καθορίζεται από την παράμετρο mode — η οποία μπορεί να είναι "greedy", "non-greedy", "post-disturbance", και μεταβιβάζεται στη perform-hist-modification.

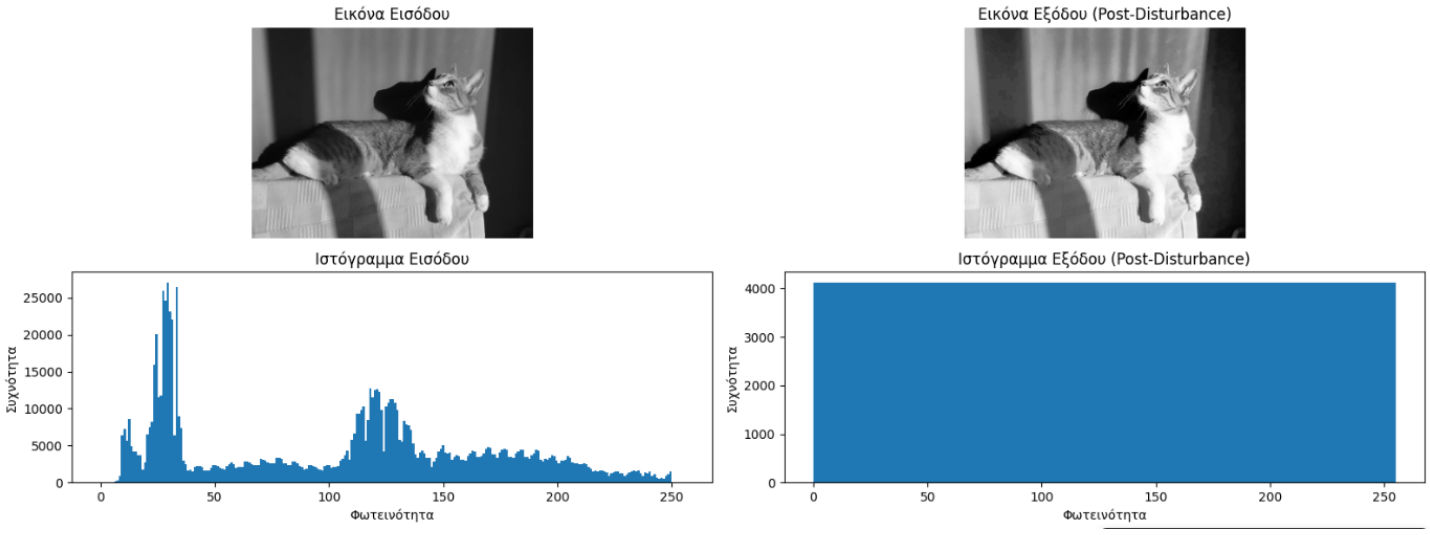
```python
def perform_hist_eq(img_array: np.ndarray, mode: str) -> np.ndarray:


    # Δημιουργία ισοκατανεμημένου histogram αναφοράς
    total_pixels = img_array.size
    uniform_count = total_pixels // 256
    remainder = total_pixels % 256

    # Δημιουργούμε ένα λεξικό που λέει πόσα pixels θέλουμε σε κάθε στάθμη (ισοκατανεμημένα)
    hist_ref = {i: uniform_count for i in range(256)}
    for i in range(remainder):
        hist_ref[i] += 1  # μοιράζουμε το υπόλοιπο



    # Instead of dividing the dictionary, normalize each value within it
    hist_ref = {k: v / total_pixels for k, v in hist_ref.items()}

    # Χρήση της perform_hist_modification σωστά
    modified = perform_hist_modification(img_array, hist_ref, mode=mode)

    # Επαναφορά στο [0,1] float
    return (modified).astype(np.float32)
```











GREEDY

Τι βλέπουμε: Το ιστόγραμμα εξόδου εμφανίζει ¨κενά' και έντονες αιχμές (spikes), ενώ παρατηρούνται εναλλαγές μεταξύ υψηλών και χαμηλών bars.

Τι συμβαίνει: Επειδή η greedy προσέγγιση αντιστοιχίζει ολόκληρες στάθμες εισόδου σε μία στάθμη εξόδου, πολλές τιμές συγκεντρώνονται σε λίγα bins. Αυτό οδηγεί σε μη ομοιόμορφη κατανομή και ¨σκαλοπάτια' στο ιστόγραμμα.

Συμπέρασμα: Η προσέγγιση είναι απλή, αλλά δεν επιτυγχάνει καλή ισοστάθμιση — ειδικά όταν το αρχικό ιστόγραμμα είναι έντονα μη ισοκατανεμημένο.

NON-GREEDY

Τι βλέπουμε: Το ιστόγραμμα εξόδου διατηρεί πολλά από τα χαρακτηριστικά του αρχικού histogram, αλλά εμφανίζεται περισσότερη ομαλότητα και εξομάλυνση σε σχέση με την greedy μέθοδο.

Τι συμβαίνει: Η non-greedy προσέγγιση χρησιμοποιεί το κριτήριο deficiencyκαι κάνει πιο ευέλικτη αντιστοίχιση, αποφεύγοντας την υπερφόρτωση μιας στάθμης εξόδου.

Συμπέρασμα: Έχει βελτιωμένο αποτέλεσμα σε σχέση με την απλή greedy μέθοδο, αλλά εξακολουθεί να είναι περιορισμένη από το ότι κάθε στάθμη εισόδου αντιστοιχίζεται ολόκληρη σε μία έξοδο.

POST-DISTURBANCE

Τι βλέπουμε: Το ιστόγραμμα εξόδου είναι σχεδόν τέλεια επίπεδο. Όλες οι στάθμες εξόδου έχουν ίδιο αριθμό pixels.

Τι συμβαίνει: Με την προσθήκη jitter, οι στάθμες εισόδου ¨σπάνε¨, επιτρέποντας πλέον την ομοιόμορφη κατανομή των pixels σε όλα τα bins μέσω sorted greedy κατανομής.

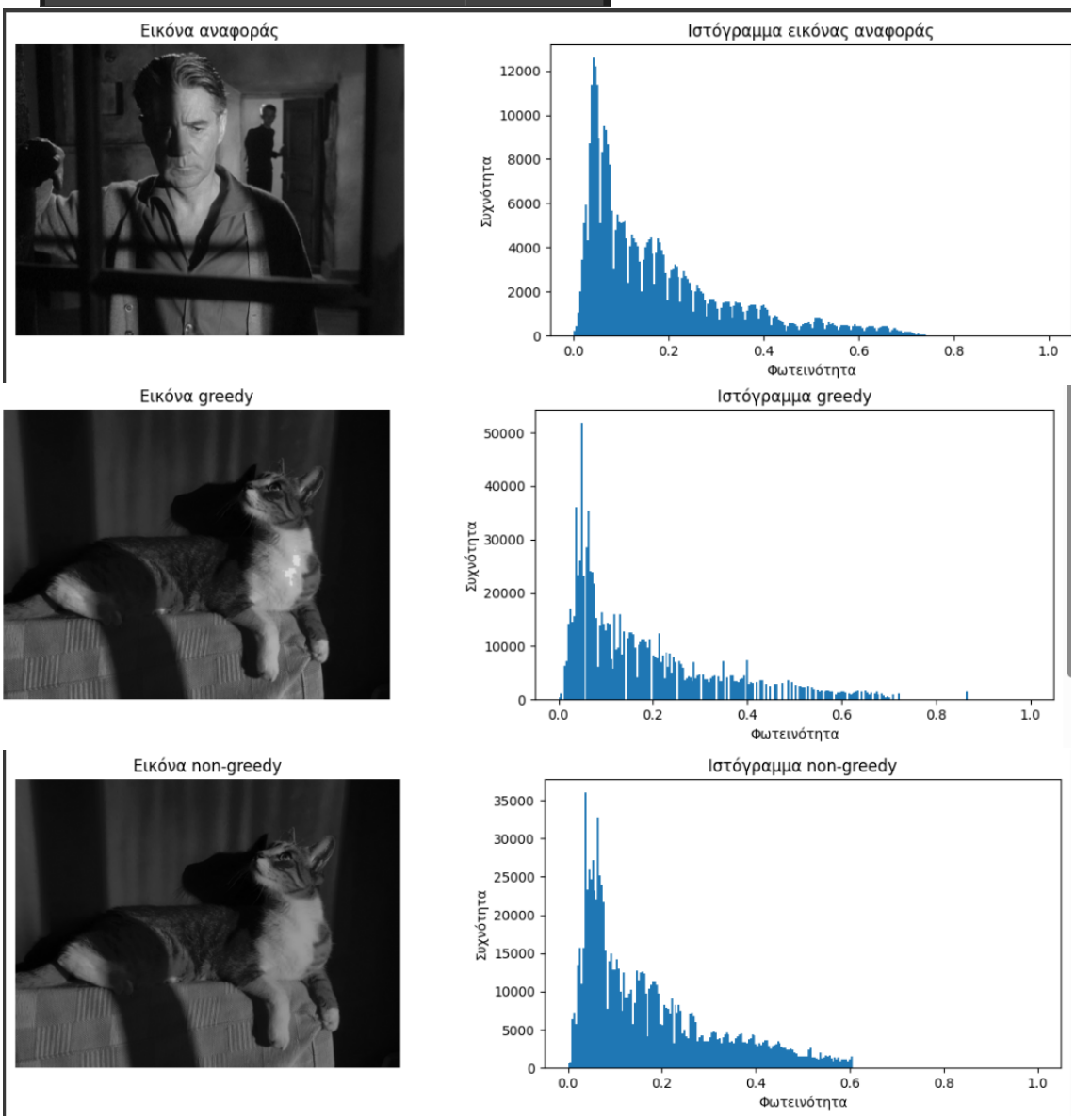
Συμπέρασμα: Αυτή η μέθοδος είναι η μόνη που επιτυγχάνει πραγματικά ισοκατανεμημένο histogram, και είναι απολύτως συμβατή με τη θεωρία της εξισορρόπησης ιστογράμματος.

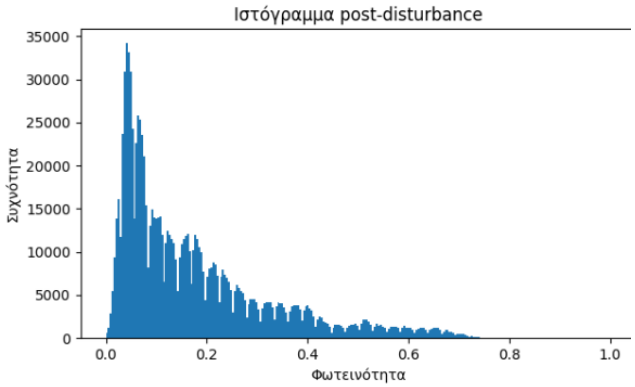
perform-hist-matching

Η συνάρτηση perform-hist-matching εφαρμόζει αντιστοίχιση ιστογράμματος (histogram matching) μεταξύ δύο εικόνων: μιας εισόδου img-array και μιας εικόνας αναφοράς img-array-ref, της οποίας το ιστόγραμμα θέλουμε να προσεγγίσουμε.

Ο τρόπος αντιστοίχισης καθορίζεται από την παράμετρο mode, που επιλέγει μία από τις τρεις διαθέσιμες μεθόδους: "greedy", "non-greedy", "post-disturbance". Η συνάρτηση χρησιμοποιεί εσωτερικά τη perform-hist-modification για να εφαρμόσει τον κατάλληλο μετασχηματισμό.

```python
def perform_hist_matching(img_array: np.ndarray, img_array_ref: np.ndarray, mode: str) -> np.ndarray:
    """
    Ορίσματα:
        • img_array: ένας διοδιάστατος numpy array τύπου dtype=float,
                     που αναπαριστά μια grayscale εικόνα εισόδου
        • img_array_ref: ένας διοδιάστατος numpy array τύπου dtype=float,
                         που αναπαριστά μια grayscale εικόνα
                         της οποίας το ιστόγραμμα καλείται να προσεγγίσει η τελική επεξεργασμένη εικόνα
        • mode: η προσέγγιση που θα πρέπει να χρησιμοποιηθεί για τη διαδικασία αντιστοίχισης ιστογράμματος.
                Δυνατές τιμές: "greedy", "non-greedy", "post-disturbance"

    Έξοδοι:
        • processed_img: ένας διοδιάστατος numpy array τύπου dtype=float, που αναπαριστά την εικόνα εξόδου

    Λειτουργία:
        η συνάρτηση δέχεται την εικόνα εισόδου f, και την εικόνα αναφοράς fref
        και καλεί κατάλληλα τη συνάρτηση perform_hist_modification(.).
    """
    # 1. Υπολογισμός του ιστογράμματος της εικόνας αναφοράς
    ref_hist = calculate_hist_of_img(img_array_ref, return_normalized=True)

    # 2. Κλήση της perform_hist_modification με το ιστόγραμμα αναφοράς
    processed_img = perform_hist_modification(img_array, ref_hist, mode)

    plt.figure(figsize=(12, 4))
    plt.hist(processed_img.flatten(), bins=256, range=(0, 1))
    plt.title(f"Ιστογραμμα {mode}")
    plt.show()

    return processed_img
```





Η σύγκριση των τριών προσεγγίσεων histogram matching (greedy, non-greedy, post-disturbance) δείχνει ξεκάθαρα διαφορά στην ποιότητα προσέγγισης της εικόνας αναφοράς. Η greedy μέθοδος αντιστοιχίζει κάθε στάθμη εισόδου μονοσήμαντα σε μία στάθμη εξόδου, με αποτέλεσμα ένα ιστόγραμμα εξόδου που περιέχει απότομα spikes και κενά, αποτυγχάνοντας να αποδώσει ομαλά την κατανομή στόχο. Η non-greedy μέθοδος, βασισμένη στο κριτήριο deficiency, επιτρέπει πιο ευέλικτη εκχώρηση των εισόδων και οδηγεί σε πιο λεία και ρεαλιστική προσέγγιση του επιθυμητού histogram, αν και ακόμα περιορίζεται από την ακεραιότητα των εισόδων. Η post-disturbance προσέγγιση προσθέτει τυχαίο jitter στις τιμές της εικόνας και διανέμει ισομερώς τα pixels μέσω ταξινόμησης, επιτυγχάνοντας εξαιρετικά ομαλή και συνεχόμενη προσέγγιση του ρεφερενςε ηιστογραμ. Οπτικά, μόνο η post-disturbance μέθοδος αναπαράγει σχεδόν πιστά τη στατιστική κατανομή της εικόνας αναφοράς, κάνοντάς την την πλέον κατάλληλη όταν ζητείται ακρίβεια και υψηλή ποιότητα αντιστοίχισης.